

Summary

Regular Expressions provide a formal mechanism for defining regular languages in Theory of Computation. The lecture highlights the following core insights:

- **Constraint Complexity:** “At least one” conditions require accommodating all valid permutations to avoid imposing unintended ordering constraints.
- **Equivalence and Redundancy:** Many complex expressions simplify to $(a+b)^*$ or $(a+b)^+$. A key heuristic is verifying whether the inner expression of a starred form can generate all base symbols and ϵ .
- **Language Complementation:** Negated constraints are often best handled by constructing the FA, complementing it, and then deriving the RE.
- **Redundancy Patterns:** Any expression already covered by a Kleene star elsewhere in the RE is redundant (e.g., $a^* + a^* = a^*$).

1. Construction Strategies for Specific Language Constraints

Careful analysis of minimum strings and flexible placements is essential in RE construction.

At Least One ‘a’ and One ‘b’

The language must include strings containing both symbols in any order.

Invalid Expression:

$(a+b)^* ab (a+b)^*$

Reason:

Cannot generate “ba”.

Valid Expression:

$(a+b)^* ab (a+b)^* + (a+b)^* ba (a+b)^*$

This accounts for both possible orders.

Alternative Valid Form:

$b^+ a^+ b^* (a+b)^* + a^+ b^+ a^* (a+b)^*$

This guarantees at least one occurrence of each symbol using positive closure.

Number of ‘a’s Followed by Number of ‘b’s

Language:

$L = \{ a^m b^n \mid m, n \geq 0 \}$

Standard Regular Expression:

$a^* b^*$

Redundant Variations:

$a^* a^* b^*$

$a^* b^* b^*$

Extra leading or trailing a^* or b^*

All reduce to:

$a^* b^*$

Invalid Construction:

$(a^* b^*)(a^* b^*)$

Reason:

Could generate “ba”, violating the required ordering constraint.

2. Logical Identities and Equivalence

A major skill in TOC is determining whether two REs define the same language.

Universal Language Identities

If an expression is of the form $(R)^*$, and R can generate a, b, and ϵ , then:

$(R)^* = (a+b)^*$

Expression	Equivalent To	Logic
$(a^* + b^*)^*$	$(a+b)^*$	Can generate ϵ , a, and b inside the star.
$(a^* b^*)^*$	$(a+b)^*$	Allows arbitrary mixing of a and b.
$(\epsilon + a + b)^+$	$(a+b)^*$	ϵ inside positive closure acts like star.
$(a+b)^* + (a+b)^*$	$(a+b)^*$	Redundant union of identical sets.

Specific Identity Rules

$R + \emptyset = R$

$R \cdot \emptyset = \emptyset$

$R \cdot \epsilon = R$

$(R^+)^+ = R^+$

$a(a+b)^* + b(a+b)^* + \epsilon = (a+b)^*$

The last identity partitions the total language by starting symbol.

3. Negation and Prefix Constraints

Handling Negation

For constraints like “does not start with a”:

Procedure:

1. Construct FA for “starts with a”.
2. Complement final and non-final states.
3. Include ϵ if the start state becomes final after complementation.

Resulting Regular Expression:

$\epsilon + b(a+b)^*$

Prefix Constraint: $|n_a(P) - n_b(P)| \leq 1$

Condition:

For every prefix P, the absolute difference between counts of a and b is at most one.

Implication:

Consecutive identical symbols are forbidden.

Reason:

“aa” produces prefix difference 2.

“bb” produces prefix difference 2.

Valid strings must alternate:

abab..., baba..., etc.

4. Programming Language Identifiers

Identifiers in programming languages follow a structured pattern.

Rule:

A letter must appear first, followed by letters or digits.

Formal Regular Expression:

$L(L+D)^*$

Equivalent Notation:

$L(L|D)^*$

Where:

L = set of letters

D = set of digits

This enforces correct lexical structure.

5. Synthesis of GATE Problem Solving

GATE 2016: Two Consecutive 0s and Two Consecutive 1s

Correct Regular Expression:

$$(0+1)^* 00 (0+1)^* 11 (0+1)^* \\ + \\ (0+1)^* 11 (0+1)^* 00 (0+1)^*$$

This can be factored using $(0+1)^*$ as a common prefix/suffix.

GATE 2004: Equivalence to $(a+b+c)^*$

Key test:

Does the inner expression generate a, b, and c?

$(a^* b^* c^*)^*$ is equivalent to $(a+b+c)^*$.

Expressions missing any base symbol are not equivalent.

GATE 1998: String Membership (11001)

Evaluation required checking:

- Can the RE generate starting “11”?
- Can it generate “00” in the middle?
- Does ordering violate internal constraints?

Membership testing requires verifying construction feasibility, not pattern matching alone.

GATE 2023: Identifiers

Correct DFA/RE model:

- Initial transition must be a letter.
- Subsequent state must loop on letters and digits.

This corresponds to:

$L(L+D)^*$